P4. Assignment 4.1 Zip Cracker in Python

Protected zipfiles cracker

Objective

The objective of this lab is to create a PythonLinks to an external site. script which automates a dictionary attackLinks to an external site. over password protected zip files.

The resolution of this lab is important to thoroughly understand the importance of using secure passwordsLinks to an external site. in any system or application. The lab will show how a very simple code could implement a password cracker that can achieve a speed of more that 5,000 password attempts per second.

Setup

You can work with any system that has python3Links to an external site. installed, and if you want, you can also use the same Linux system that you have been using in this class.

Creating the Script

The script should work by basically accepting two parameters:

- The protected zipped file that you want to crack
- The dictionary text file that contains the list of words that you want to try against the zipfile.

We are storing your individual zipped-password-protected-files **in your home directory of the master server "/home/<username>/lab4.1".**
> Remember that Master Server connection goes over port 10010 (ssh username@cy5001.mscy.org -p 10010)

```
@cy5001:/home/<username>/lab4.1$ ls -la
total 28
drwxr-xr-x 2 root    root 4096 Apr  2 10:37 .
drwxr-x--- 4 <username> ta   4096 Apr  2 10:37 ..
-rw-r--r-- 1 root    root  312 Apr  2 10:37 <username>.1.zip
```

```
-rw-r--r-- 1 root    root  311 Apr  2 10:37 <username>.2.zip
-rw-r--r-- 1 root    root  310 Apr  2 10:37 <username>.3.zip
-rw-r--r-- 1 root    root  311 Apr  2 10:37 <username>.4.zip
-rw-r--r-- 1 root    root  311 Apr  2 10:37 <username>.5.zip
```

You must use a particular dictionary file with tentative passwords, which you can download form here: [https://users.cs.duke.edu/~ola/ap/linuxwordsLinks to an external site.](https://users.cs.duke.edu/~ola/ap/linuxwords) .

You are free to create other zipped files as well in order to test your system.

[ccl-user@home ~]$ zip -e zipped.zip zipme.txt

Enter password:

> You'll be prompted to select the password you want to use

So, there are many of dictionary files available on the internet across different websites. Ensure that you download files from trusted and legitimate sources.

[!!!] Warning! There are malicious content that can be inadvertently downloaded when visiting Web Sites!!

> The output of your script should look something like this.

$ [ccl-user@home ~]$ ./cracker.py zipped.zip wordlist.txt

```
.....

[------- Password  Found ---------] --> canvas


Passwords attempted: 481265.
```

Basically, your script should have the following sections:

- Start with the location of the python3 binary file (it may change)
    - -#!/usr/bin/python3
- Import libraries

- - import zipfile # Library to extract zip files
- - import time # Library to measure the time spent in the cracking process
- - import sys # Library to access the arguments of the script

- Load the provided arguments into local variables and open the zipfile and the dictionary file
  - - zfile = sys.argv[1]
  - - dfile = sys.argv[2]
  - - filezip = zipfile.ZipFile(zfile) # We used the "zipfile" library to open the zipped file.
  - - passFile = open(dfile) # opening the dictionary file.
- Start the for loop which will try all tentative passwords present in the dictionary file.
  - for line in passFile.readlines():

```
    password = line.strip("\n")
 # removing the end of line
    codedpass = bytes(password, 'utf-8')
 # password encoded in binary
   count = count + 1
   try:
# trying the password using "zipfile" library
      filezip.extractall(pwd=codedpass)
  except:
# if the password was not successful, then print it
      print (" ** Attempted ** " + password)
  else:
      print ("[------- Password Found ---------] --> " + password)
      print ("Passwords attempted: " + str(count))
      exit(0)
```

- And finally, if the password was not found, print how many passwords were attempted.
    - print ("[-] Password not found. Attempted: " + str(count)

Task 1: Create the script and crack the 5 zipped files provided

In this task you need to create the cracking script by following the recommendations and structure that we have provided to you above. If you decide to approach the problem in a different way, you are free to do it, just make sure that you comment and structure your script in a way that it is easy to understand.

Don't forget that in order to execute a script, you need to provide execution privileges in Linux.

chmod +x crack.py

The zip file provided were created using passwords included into the dictionary text file provided, so if you use this specific dictionary you will be able to crack them all.

```
jose$ ./zipcracker.py pass2.zip dictionary.txt
.....
 [------- Password Found ---------] --> NADA
```

<u>Submit a text file name <username>.flags4.1.txt with the flags (1 flag per line) that you have discovered as the content of the zipped files</u>

$cat <username>.flag4.1.txt

f238b69cea2073574e6f757bf1c08f1cad4ccf720e73dc961400a546f8ba27556ddc8
dbe0bcb49db7ad7119f17f348dd4ba6fb390a558d3d8971e5d390a558d3
cd0975b31dd42fcd8be532a5aade77933516fa0322b7889a12117cf9b0b3d80c8dc
b852574b0ba99c7a14f6a07d825219feee31c818e6bbfbe7dcd260e7b2d5d

24406e19b77f0b017f7390a558d3d8971e5db66e3a8ce87a610790bc0509d82439a
bc3c072a685ed548f903993d583212c49144722cf509ac707bf10f348dd4b
38e29f03ae5fcda691888f2acde7486a4582165f6619de29930cf496748bf6e2159bf
0c6d934ddb0cec841e42b374934e7b57ece1e60b624ef1535c9e2acf2e1
4b59ec8d823145226e0dc5f58cb3cb61661ad2462bd44df21c9d3caf9094ccfd4843
c214e2e18509eafe0085fa2b83e8096cdb20f808ad595e0eab5aae029a5d

Task 2: Improve the script providing time spent and cracking speed.

In this case you need to improve the script to be able to track how long does it takes to crack the zipfile (by printing cracking rate and time elapsed).

This is an example of the improved output of the script.

$ [ccl-user@home ~]$ ./cracker.py zipped.zip wordlist.txt

.....

[------- Password Found ---------] --> canvas

Time consumed: 85.31591200828552 seconds

Passwords attempted: 481265. At 5640.975858679933 tries per second

Task 3: Improve cracking speed

In this task you are required to find a way to improve cracking speed in your script. There are several approaches you can go, but we recommend you to simply reduce input/output operations.

Cracking rate BEFORE improvements

[------- Password Found ---------] --> monicat

Time consumed: 85.60599684715271 seconds

Passwords attempted: 481270. At 5621.919231421311 tries per second

Cracking rate AFTER improvements

[------- Password Found ---------] --> monicat

Time consumed: 52.617936849594116 seconds

Passwords attempted: 481270. At 9146.500771698586 tries per second

Submit your resulting script named **<username>.zipcracker.py**

Task 4: Tailor a wordlist with Regular expression

In this case, instead of using a previously created password list, you need to create a password list based on regular expression. Let's imagine that the password you need to crack is based on a list of family names in lowercase (peter, belinda, mario and pooja) and a date from 01-01-2000 to 12-31-2020 (mmddyy format).

So a few examples of passwords will be : mario110915, peter010101, belinda090809, pooja043020, ...

You must use the following python library [EXREXLinks to an external site.] to create that password list. Include in your report clearly the regular expression that you have used and provide the file **<username>.task4.list** with the dictionary of passwords taht you have generated.

Your submission for task4 need to include a zipped and password-protected file that you create to test it: "**<username>.task4.zipper"**.

>> This zipped file has been created protected using a randomly selected password from the **<username>.task4.list.**

Submission

In this assignment, you'll need to submit to canvas the following deliverables:

1. **File with the flags <username>.flags4.1.txt**
2. **File with your final zipcracker script (the one with the better performance, Task 3) <username>.zipcracker.py**
3. **Your assignment report <username>.report4.1.pdf**
   + Include in your report **the EXREX command you have used to create your task4 password list** (**<username>.task4.list**)

+ Include **the password you used to encrypt the task 4 zipped** (**<username>.task4.zipper)**

4. **Make sure the you Rename the <username>.task4.zip into <username>.task4.zipper** before submitting it.
5. **File <username>.task4.list**

Please make sure to test your script in an empty folder, different from the one you have been using for your work (so, it will look like the folder where we will execute it to evaluate it).

Grading

Please see here the rubric for the assignment:

Concept	**Points**
Task 1	30 (Gradescope check the flags)
Task 2	30 (Gradescope manually graded, pdf report)
Task 3	20 (Gradescope manually graded, pdf report)
Task 4	20 (Gradescope check the zipped file and the dictionary)

Although gradescope provides a grading for the task that can be automatically graded, these grades can be reviewed manually.